MATH3881/5881: Statistical Machine Learning Theory

Week 4: Optimisation in Machine Learning: Algorithms

Sarat Moka *UNSW, Sydney*

Key Topics

Books:

(A) *Mathematical Engineering of Deep Learning* book by Liquet, Moka, and Nazarathy (2024), Chapter 4.

4.1 Stochastic Gradient Descent

In Week 3 we proved that gradient descent (GD) converges at rate $O(1/t)$ for convex smooth objectives, and geometrically for strongly convex ones. However, to achieve this rate, each GD step requires computing the *full* gradient

$$\nabla C(\theta) = \frac{1}{n}\sum_{i=1}^{n} \nabla C_i(\theta),$$

which costs $O(nd)$ per iteration, intractable when n is large; here recall that $d = |\theta|$ is the number of parameters in the network (i.e., weights and biases). The remedy is to use cheap, noisy gradient approximations.

4.1.1 SGD and Mini-batch SGD

Stochastic gradient descent (SGD) selects an index I_t uniformly at random from $\{1, \dots, n\}$ and updates

$$\theta^{(t+1)} = \theta^{(t)} - \alpha\, \nabla C_{I_t}(\theta^{(t)}). \tag{4.1}$$

Each step costs $O(d)$, a factor of n cheaper than full GD. The key property is **unbiasedness**: for fixed θ,

$$\mathbb{E}\big[\nabla C_{I_t}(\theta)\big] = \frac{1}{n}\sum_{i=1}^{n} \nabla C_i(\theta) = \nabla C(\theta). \tag{4.2}$$

Hence each SGD step is "on average correct," but individual steps introduce variance (noise).

Mini-batch SGD uses a random subset $\mathcal{I}_t \subseteq \{1, \dots, n\}$ of size n_b and replaces the gradient with

$$\hat{g}^{(t)} = \frac{1}{n_b}\sum_{i \in \mathcal{I}_t} \nabla C_i(\theta^{(t)}). \tag{4.3}$$

This is again unbiased: $\mathbb{E}[\hat{g}^{(t)}] = \nabla C(\theta^{(t)})$. For a random vector $\xi \in \mathbb{R}^d$, write $\mathrm{Var}(\xi) := \mathbb{E}[(\xi - \mathbb{E}\xi)(\xi - \mathbb{E}\xi)^\top]$ for the $d \times d$ matrix whose (i, j) entry is $\mathbb{E}[(\xi_i - \mathbb{E}\xi_i)(\xi_j - \mathbb{E}\xi_j)]$. Moreover, if the indices are sampled independently with replacement, the variance matrix satisfies

$$\mathrm{Var}\Big(\hat{g}^{(t)}\Big) = \frac{1}{n_b}\mathrm{Var}\Big(\nabla C_{I_{t,1}}(\theta^{(t)})\Big), \tag{4.4}$$

so each entry of $\mathrm{Var}(\hat{g}^{(t)})$ is n_b times smaller than for single-sample SGD.

Theory vs practice: IID sampling vs random reshuffling. The variance formula (4.4) is exact under *IID-with-replacement* sampling, where I_t (or each element of $\mathcal{I}_t$) is drawn fresh and uniformly from $\{1, \dots, n\}$ at every iteration. This is the regime our analysis uses. In practice, however, modern training frameworks (e.g. PyTorch's `DataLoader(shuffle=True)`) implement **random reshuffling** instead: shuffle the data once, divide it into $\lfloor n/n_b \rfloor$ disjoint mini-batches,

iterate through them sequentially, then reshuffle for the next pass. A full pass over all $\lfloor n/n_b \rfloor$ mini-batches is called an **epoch**. Reshuffling guarantees each sample is seen exactly once per epoch and prefetches well on GPUs, at the cost that the within-epoch indices are no longer independent, so (4.4) no longer holds exactly (the order $1/n_b$ is preserved). The IID model gives the cleanest theoretical results; the convergence rate cited in §4.1.3 extends to random reshuffling at the same order.[1]

Exercise 4.1.1 (Variance under without-replacement sampling)

Fix $\theta \in \mathbb{R}^d$, write $g_i := \nabla C_i(\theta)$ and $\bar{g} := \frac{1}{n}\sum_{i=1}^n g_i = \nabla C(\theta)$, and let $\Sigma := \mathrm{Var}(g_{I_{t,1}})$ when $I_{t,1}$ is uniform on $\{1,\dots,n\}$ (so $\Sigma = \frac{1}{n}\sum_i (g_i - \bar{g})(g_i - \bar{g})^\top$). Suppose $\mathcal{I}_t = (I_{t,1},\dots,I_{t,n_b})$ is a uniform random subset of size n_b drawn *without replacement* within a single mini-batch. Show that the mini-batch gradient $\hat{g}^{(t)} = \frac{1}{n_b}\sum_{i\in\mathcal{I}_t} g_i$ satisfies

$$\mathrm{Var}(\hat{g}^{(t)}) \;=\; \frac{1}{n_b}\cdot\frac{n-n_b}{n-1}\,\Sigma,$$

i.e. the IID variance formula (4.4) multiplied by the *finite-population correction* $(n-n_b)/(n-1)$.
Hint. For random vectors $\xi,\eta\in\mathbb{R}^d$, let $\mathrm{Cov}(\xi,\eta) := \mathbb{E}[(\xi-\mathbb{E}\xi)(\eta-\mathbb{E}\eta)^\top]$. Expand

$$\mathrm{Var}(\hat{g}^{(t)}) \;=\; \frac{1}{n_b^2}\sum_{j,k=1}^{n_b}\mathrm{Cov}\big(g_{I_{t,j}},\, g_{I_{t,k}}\big).$$

The n_b diagonal terms ($j=k$) each contribute Σ. For $j\neq k$, use $\mathbb{P}(I_{t,j}=a,\, I_{t,k}=b) = 1/[n(n-1)]$ for $a\neq b$ to show $\mathrm{Cov}(g_{I_{t,j}}, g_{I_{t,k}}) = -\Sigma/(n-1)$. There are $n_b(n_b-1)$ such off-diagonal terms; combine.

4.1.2 An illustrative example: GD vs SGD on a finite-sum quadratic

To build geometric intuition for the difference between GD and SGD, consider the following finite-sum objective with $d = |\theta| = 2$. For each sample $i\in\{1,\dots,n\}$, set

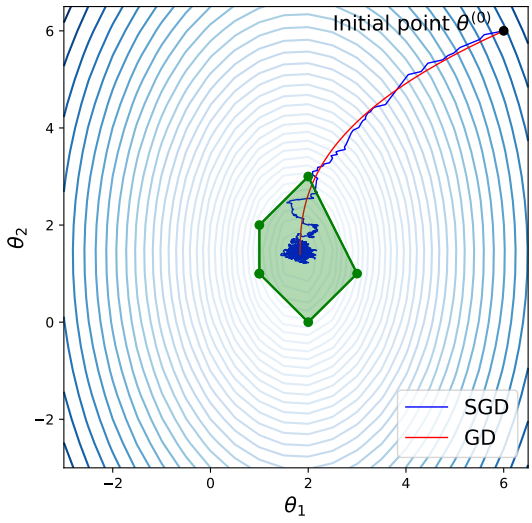
$$C_i(\theta) \;=\; a_i\,(\theta_1 - u_{i,1})^2 \;+\; (\theta_2 - u_{i,2})^2, \tag{4.5}$$

for constants $a_i > 0$ and points $u_i = (u_{i,1}, u_{i,2}) \in \mathbb{R}^2$. Each C_i is convex with unique minimiser $u_i = (u_{i,1},u_{i,2})^\top$, so the total objective $C(\theta) = \frac{1}{n}\sum_{i=1}^n C_i(\theta)$ is also convex. Let

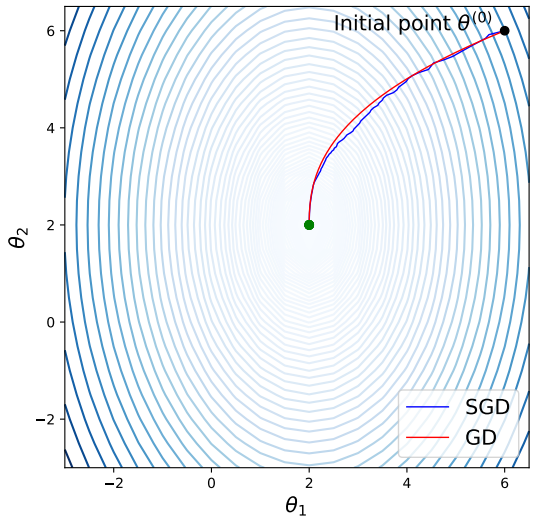
$$\mathcal{S} \;:=\; \mathrm{conv}\{u_1,\dots,u_n\} = \Big\{\sum_{i=1}^n \lambda_i u_i \;:\; \lambda_i\geq 0,\; \sum_{i=1}^n \lambda_i = 1\Big\}$$

denote the convex hull of the per-sample minimisers. Because each $-\nabla C_i(\theta)$ points toward its minimiser $u_i\in\mathcal{S}$, the global minimiser θ^* of C also lies in $\mathcal{S}$.

[1] Mishchenko, K., Khaled, A., & Richtárik, P. (2020). Random reshuffling: Simple analysis with vast improvements. *Advances in Neural Information Processing Systems (NeurIPS)* 33.



(a) Distinct per-sample minimisers $u_1, \ldots, u_5$ (green dots), so $\mathcal{S}$ is a non-trivial polygon. Outside $\mathcal{S}$ both trajectories head into $\mathcal{S}$ along similar paths; once inside, SGD bounces around while GD continues smoothly to θ^*.



(b) Coincident per-sample minimisers ($u_i = u$ for all i, so $\mathcal{S} = \{u\}$). All $-\nabla C_i$ agree everywhere, so SGD reduces to GD with rescaled gradient and there is essentially no fluctuation.

Figure 4.1: GD vs SGD trajectories on the finite-sum quadratic (4.5) with $n = 5$. The shaded region is $\mathcal{S} = \text{conv}\{u_1, \ldots, u_n\}$. Outside $\mathcal{S}$, GD and SGD move similarly; inside $\mathcal{S}$, SGD fluctuates because the per-sample gradients ∇C_i disagree about the descent direction.

Why both methods are pulled toward $\mathcal{S}$. At any point $\theta \notin \mathcal{S}$, the per-sample steepest-descent direction $-\nabla C_i(\theta)$ points into $\mathcal{S}$, for every i. Therefore:

- The *averaged* direction $-\nabla C(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \nabla C_i(\theta)$, used by GD, points into $\mathcal{S}$.
- The *random* direction $-\nabla C_{I_t}(\theta)$, used by SGD, also points into $\mathcal{S}$ regardless of which index I_t is sampled.

So outside $\mathcal{S}$ the SGD trajectory looks essentially like the GD trajectory, with only mild noise around a directionally-correct step. Figure 4.1 shows the resulting trajectories on a concrete instance with $n = 5$.

Why SGD fluctuates inside $\mathcal{S}$. Once $\theta^{(t)} \in \mathcal{S}$, the per-sample gradients $-\nabla C_i(\theta^{(t)})$ point in different directions: each is pulling toward its own minimiser u_i. The averaged direction $-\nabla C(\theta^{(t)})$ still points toward θ^*, so GD continues to descend smoothly. But SGD picks one of those disagreeing directions at random at each step, so its trajectory bounces among the u_i and fluctuates around (but largely stays inside) $\mathcal{S}$. We can reduce these fluctuations using mini-batches; in the limiting case

$n_b = n$, the mini-batch coincides with the full dataset, so SGD reduces to GD and the fluctuations vanish entirely.

This example is hypothetical (real deep learning losses are highly non-convex with many local minima), but it captures the essential trade-off: SGD's per-step direction is correct *on average* but noisy on any individual step, with the fluctuations concentrated in the region of parameter space where the per-sample minimisers live.

4.1.3 Convergence of SGD

For full GD on a convex L-smooth objective, we proved the $O(1/t)$ rate in Week 3. SGD's noise prevents the same rate: the best achievable rate is $O(\log T/\sqrt{T})$, attained with a *diminishing* step size $\alpha_t = c/\sqrt{t}$. Mini-batches reduce the constant in front of this rate (via the variance formula (4.4)) but do not change the order. Under suitable assumptions, this $1/\sqrt{T}$ scaling is unavoidable for stochastic first-order methods.[2]

Cost comparison with full GD. By the Week 3 convergence theorem, GD on a convex objective reaches suboptimality ε in $O(1/\varepsilon)$ iterations, each at cost $O(nd)$, for total cost $O(nd/\varepsilon)$. SGD reaches the same accuracy *in expectation* in $O(1/\varepsilon^2)$ iterations, each at cost only $O(d)$, for total cost $O(d/\varepsilon^2)$. SGD wins whenever $\varepsilon \gtrsim 1/n$, which is precisely the regime relevant to deep learning: n is huge and only moderate accuracy is needed.

Remark 4.1.1

SGD noise as a feature. The noisy gradient in (4.1) causes the trajectory to fluctuate, which helps escape saddle points and local minima in non-convex landscapes. This is one reason SGD and mini-batch SGD often outperform full GD in deep learning practice despite the worse convergence rate guarantee. Further, it has been observed empirically that SGD's noise has an *implicit regularisation* effect that improves generalisation.

4.2 Momentum and Nesterov Acceleration

Basic gradient descent has no memory: each step uses only the current gradient, so in elongated valleys it oscillates across the narrow direction while creeping along the flat one. **Momentum** accumulates a velocity vector from past gradients to dampen the oscillation and build up speed along the persistent direction.

[2]Standard treatments: Bubeck, S. (2015). *Convex Optimization: Algorithms and Complexity.* Foundations and Trends in Machine Learning 8(3–4), 231–357; Bottou, L., Curtis, F. E., & Nocedal, J. (2018). Optimization methods for large-scale machine learning. *SIAM Review* 60(2), 223–311.

4.2.1 Classical Momentum

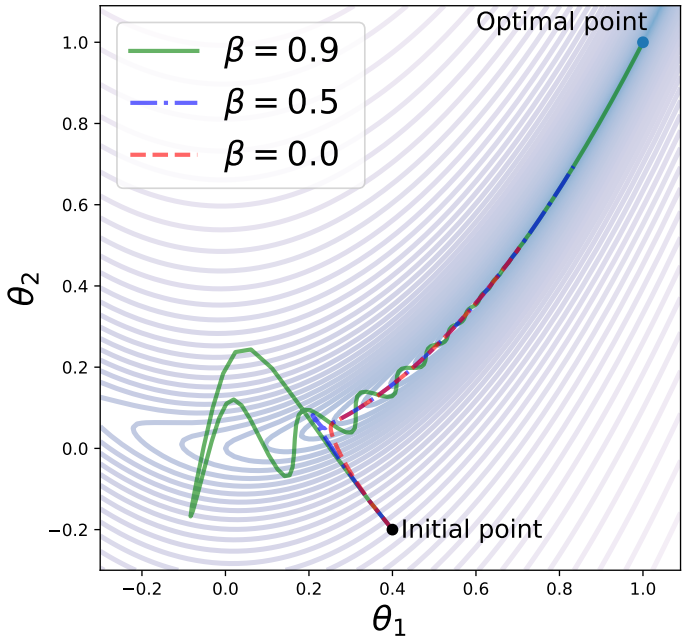


Figure 4.2: Momentum applied to the Rosenbrock function for three values of β (with $\alpha = 0.001/(1-\beta)$). At $\beta = 0$ this is basic GD; larger β enables faster progress along the valley.

The momentum method maintains an exponentially smoothed gradient vector $v^{(t)}$, called the **momentum**, and uses it as the descent direction:

$$v^{(t+1)} = \beta\, v^{(t)} + (1-\beta)\, \nabla C(\theta^{(t)}) \tag{4.6}$$

$$\theta^{(t+1)} = \theta^{(t)} - \alpha\, v^{(t+1)}, \tag{4.7}$$

with $v^{(0)} = 0$, momentum parameter $\beta \in [0, 1)$, and learning rate $\alpha > 0$.[3] Unrolling (4.6):

$$v^{(t+1)} = (1-\beta) \sum_{\tau=0}^{t} \beta^{t-\tau} \nabla C(\theta^{(\tau)}), \tag{4.8}$$

so $v^{(t+1)}$ is a weighted average of all past gradients, with exponentially decaying weights. When $\beta = 0$, (4.7) reduces to basic GD. In practice for deep networks, the full-batch gradient $\nabla C(\theta^{(t)})$ in (4.6) is replaced by a stochastic mini-batch estimate; the momentum recursion itself is unchanged.

[3]Some implementations drop the $(1-\beta)$ factor in (4.6) and use $v^{(t+1)} = \beta v^{(t)} + \nabla C(\theta^{(t)})$ instead; this is the convention of PyTorch's `torch.optim.SGD(momentum=·)` and of Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the importance of initialization and momentum in deep learning. *Proceedings of the 30th International Conference on Machine Learning (ICML)*. The two forms are equivalent: rescaling $\alpha \mapsto \alpha/(1-\beta)$ recovers identical iterates.

Remark 4.2.1

The weights $(1-\beta)\beta^{t-\tau}$ in (4.8) are geometric with mean lag $\beta/(1-\beta) \approx 1/(1-\beta)$, so momentum effectively averages the last $\approx 1/(1-\beta)$ gradients. For $\beta = 0.9$ this is about 10 steps; for $\beta = 0.99$ about 100. In flat regions, momentum prevents stagnation by carrying over the kinetic energy from previous descent steps.

Exercise 4.2.1

Suppose $\nabla C(\theta^{(\tau)}) = g$ is constant for all $\tau \geq 0$, with $v^{(0)} = 0$. Using (4.8), find $v^{(t+1)}$ in closed form. What does $v^{(t+1)}$ converge to as $t \to \infty$, and what does the momentum update (4.7) reduce to in this limit?

4.2.2 Nesterov Accelerated Gradient

Nesterov's method modifies momentum by evaluating the gradient at the *look-ahead* point $\theta^{(t)} - \alpha\beta v^{(t)}$ rather than the current point $\theta^{(t)}$:

$$v^{(t+1)} = \beta\, v^{(t)} + (1-\beta) \underbrace{\nabla C\big(\theta^{(t)} - \alpha\beta v^{(t)}\big)}_{\text{gradient at look-ahead point}}, \tag{4.9}$$

$$\theta^{(t+1)} = \theta^{(t)} - \alpha\, v^{(t+1)}, \tag{4.10}$$

with $v^{(0)} = 0$. This "corrects" the momentum step by using gradient information from where we are headed, rather than where we are. The result is a dramatically improved convergence rate.

Theorem 4.2.1 (NAG Rate)

Let C be convex and L-smooth with minimiser θ^*. Nesterov's accelerated gradient method[a] with step size $\alpha = 1/L$ and an iteration-dependent schedule of the momentum parameter β (specified in the original paper) achieves

$$C(\theta^{(t)}) - C(\theta^*) \leq \frac{2L\|\theta^{(0)} - \theta^*\|^2}{(t+1)^2}. \tag{4.11}$$

[a] Nesterov, Y. (1983). A method for solving the convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady* 27(2), 372–376.

The above $O(1/t^2)$ rate of NAG versus $O(1/t)$ for GD is not merely a small improvement: for ε-accuracy, GD needs $O(1/\varepsilon)$ iterations while NAG needs only $O(1/\sqrt{\varepsilon})$. This $O(1/t^2)$ rate is *optimal* for first-order methods on convex smooth problems: no first-order algorithm can improve upon it without additional assumptions. Achieving the bound requires an iteration-dependent β schedule; the constant-β form used in practice does not, in general, attain $O(1/t^2)$.

4.3 Adaptive Learning Rate Methods

Standard GD and momentum use the same learning rate α for all coordinates of θ. In practice, different parameters may need to be updated at different scales: e.g. in NLP, the parameter for a rare word receives a non-zero gradient on only a small fraction of mini-batches, so a global α tuned for rare words overshoots on frequent ones and vice versa. **Adaptive methods** learn a per-coordinate effective learning rate from gradient history.

4.3.1 AdaGrad

AdaGrad (Adaptive Subgradient)[4] accumulates the sum of squared partial derivatives coordinate-wise (with empty-sum convention $s_i^{(0)} = 0$):

$$s_i^{(t+1)} = \sum_{\tau=0}^{t} \left(\frac{\partial C(\theta^{(\tau)})}{\partial \theta_i} \right)^2, \tag{4.12}$$

and updates each coordinate with an individual effective rate:

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \frac{\alpha}{\sqrt{s_i^{(t+1)}} + \varepsilon} \frac{\partial C(\theta^{(t)})}{\partial \theta_i}, \tag{4.13}$$

where $\varepsilon \approx 10^{-8}$ prevents division by zero. Coordinates with large historical gradients receive smaller effective rates; rarely updated coordinates receive larger effective rates, improving learning of infrequent features. At $t = 0$, $\sqrt{s_i^{(1)}} = |\partial C(\theta^{(0)})/\partial \theta_i|$, so the first step on coordinate i has magnitude $\approx \alpha$ regardless of how steep that coordinate's gradient is: AdaGrad rescales each coordinate to a common scale rather than respecting raw gradient magnitudes.

The drawback of AdaGrad is that $s_i^{(t)}$ is strictly non-decreasing, so the effective learning rate shrinks to zero over time, often before convergence.

4.3.2 RMSProp

RMSProp[5] fixes AdaGrad's diminishing rate by replacing the cumulative sum with an exponential moving average:

$$s_i^{(t+1)} = \gamma\, s_i^{(t)} + (1 - \gamma) \left(\frac{\partial C(\theta^{(t)})}{\partial \theta_i} \right)^2, \tag{4.14}$$

with decay parameter $\gamma \in [0, 1)$ (typically $\gamma = 0.999$) and $s_i^{(0)} = 0$. As in §4.2, the geometric weights $(1-\gamma)\gamma^{t-\tau}$ give an effective window of the last $\approx 1/(1-\gamma)$ steps (about 1000 for $\gamma = 0.999$). Hence

[4] Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research* 12, 2121–2159.

[5] RMSProp was introduced by Geoff Hinton in Lecture 6e of his 2012 Coursera course *Neural Networks for Machine Learning*; it has no published paper.

$\sqrt{s_i^{(t+1)}}$ is approximately the *root-mean-square* of recent gradient magnitudes on coordinate i, giving the method its name. The update formula remains (4.13), with $s_i^{(t+1)}$ now given by (4.14) rather than (4.12). In vector notation:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\alpha}{\sqrt{s^{(t+1)}} + \varepsilon} \odot \nabla C(\theta^{(t)}), \tag{4.15}$$

where $\odot$ is the element-wise (Hadamard) product and all operations on vectors are element-wise. As for momentum, the initial $s_i^{(0)} = 0$ biases early estimates toward zero; Adam (below) corrects this.

4.3.3 Adam: Adaptive Moment Estimation

Adam[6] combines momentum (exponential smoothing of gradients) with RMSProp (exponential smoothing of squared gradients), both with bias correction. It is the default optimiser for most deep learning tasks. Intuitively, if momentum is a ball rolling down a slope, Adam is a heavy ball with friction: it accumulates direction from past gradients and dampens speed by recent gradient magnitudes, with the early-iteration zero-bias removed.

Exponential smoothing and bias correction. Both momentum and RMSProp initialise at $v^{(0)} = 0$ and $s^{(0)} = 0$. Since β and γ are close to 1, early iterates are severely biased toward zero. To derive the right correction, suppose the gradient is constant, $\nabla C(\theta^{(\tau)}) = g$ for all τ; then the unrolled momentum (4.8) gives $v^{(t+1)} = (1 - \beta^{t+1})\, g$, so dividing by $(1 - \beta^{t+1})$ exactly recovers g. The same argument for s gives a $(1 - \gamma^{t+1})$ factor. Hence the bias-corrected estimates are:

$$\hat{v}^{(t+1)} = \frac{v^{(t+1)}}{1 - \beta^{t+1}}, \qquad \hat{s}^{(t+1)} = \frac{s^{(t+1)}}{1 - \gamma^{t+1}}. \tag{4.16}$$

As $t \to \infty$, $\beta^t \to 0$ and the correction disappears.

Adam update. The full Adam parameter update is:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{\hat{v}^{(t+1)}}{\sqrt{\hat{s}^{(t+1)}} + \varepsilon}. \tag{4.17}$$

[6] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*; published at ICLR 2015.

Algorithm 1: Adam (defaults: $\beta = 0.9$, $\gamma = 0.999$, $\varepsilon = 10^{-8}$)

Input: Objective $C(\cdot)$, initial parameters $\theta^{(0)}$
Output: Approximately optimal θ

$t \leftarrow 0$; $v \leftarrow 0$; $s \leftarrow 0$
repeat
  $g \leftarrow \nabla C(\theta)$ (compute full gradient or mini-batch estimate)
  $v \leftarrow \beta\, v + (1-\beta)\, g$ (momentum update)
  $s \leftarrow \gamma\, s + (1-\gamma)\, (g \odot g)$ (second moment update)
  $\hat{v} \leftarrow v/(1-\beta^{t+1})$; $\hat{s} \leftarrow s/(1-\gamma^{t+1})$ (bias correction)
  $\theta \leftarrow \theta - \alpha\, \hat{v}/(\sqrt{\hat{s}} + \varepsilon)$ (parameter update)
  $t \leftarrow t + 1$
until termination condition
return θ

Remark 4.3.1

The effective per-coordinate step size in Adam is $\alpha\, \hat{v}_i/(\sqrt{\hat{s}_i} + \varepsilon)$. Since $\hat{v}_i \approx \mathbb{E}[\nabla_i C]$ and $\sqrt{\hat{s}_i} \approx \sqrt{\mathbb{E}[(\nabla_i C)^2]}$ (root mean square of recent gradients on coordinate i), the ratio $\hat{v}_i/\sqrt{\hat{s}_i}$ normalises each update by the typical gradient magnitude in that coordinate, giving updates of roughly $\pm\alpha$ regardless of gradient scale. This makes Adam much less sensitive to the choice of α than basic GD.

Exercise 4.3.1 (Adaptive Methods: Bias Correction and Effective Rates)

(a) Consider one pass through the Adam algorithm (Algorithm 1) with $t = 0$, initial parameters $v^{(0)} = s^{(0)} = 0$, and a gradient $g = \nabla C(\theta^{(0)})$. Compute $v^{(1)}, s^{(1)}, \hat{v}^{(1)}, \hat{s}^{(1)}$ symbolically. Show that for any non-zero coordinate g_i, the resulting parameter update satisfies $|\theta_i^{(1)} - \theta_i^{(0)}| = \alpha/(1 + \varepsilon/|g_i|) \approx \alpha$. Why is this desirable at the start of training?

(b) Consider AdaGrad and RMSProp applied to a 1-d problem with constant gradient $\partial C/\partial\theta = g > 0$ at every step, starting from $s^{(0)} = 0$. Derive closed-form expressions for $s^{(t)}$ under each method. Hence show that the AdaGrad effective step size $\alpha/\sqrt{s^{(t)}}$ decays as $\alpha/(g\sqrt{t}) \to 0$, while the RMSProp effective step size converges to α/g as $t \to \infty$. (Ignore ε.)

4.3.4 Learning Rate Schedules

In practice, a fixed learning rate is rarely optimal throughout training. Any of the optimisers above (SGD, momentum, RMSProp, Adam) can be combined with a schedule by replacing the constant α with an iteration-dependent $\alpha^{(t)}$. Recall §4.1.3's diminishing schedule $\alpha_t = c/\sqrt{t}$, which delivers the optimal SGD convergence rate; the schedules below are widely-used practical analogues. Common choices:

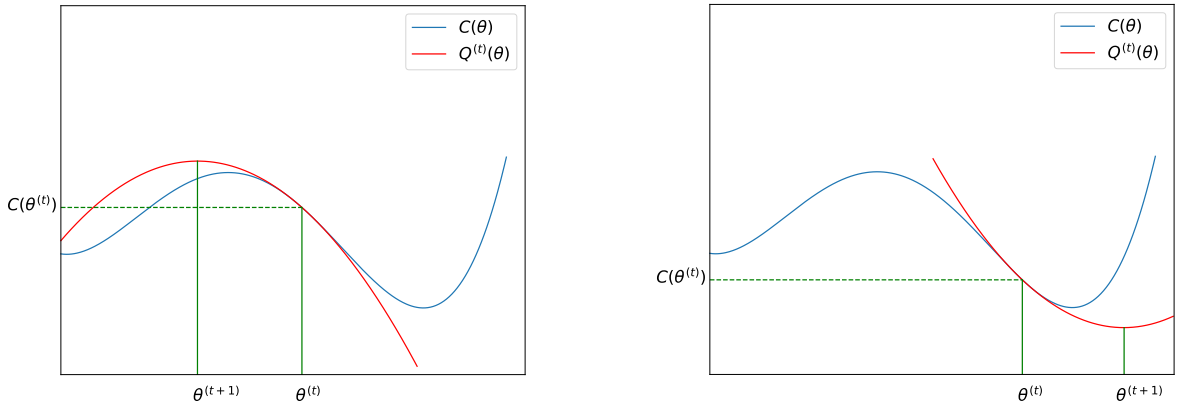
- **Step/exponential decay**: $\alpha^{(t)} = \alpha_0 \cdot r^{\lfloor t/k \rfloor}$ for decay rate $r < 1$ and period k ($k = 1$ recovers smooth exponential decay).
- **Cosine annealing**[7]: $\alpha^{(t)} = \alpha_{\min} + \frac{1}{2}(\alpha_0 - \alpha_{\min})(1 + \cos(\pi t/T))$, smoothly decaying to $\alpha_{\min}$.
- **Linear warm-up**: ramp $\alpha^{(t)}$ linearly from 0 to α_0 over the first T_{warm} steps to stabilise early training, then decay.

Warm-up is commonly used with Adam (and is standard practice when training Transformers): with only a handful of squared-gradient samples, the *variance* of $\hat{s}_i$ is very high, so a coordinate where $\hat{s}_i$ happens to be small by chance receives a hugely inflated effective rate $\alpha/\sqrt{\hat{s}_i}$. A small initial learning rate caps the early step magnitudes while $\hat{s}$ stabilises.

4.4 Second-Order Methods

All methods so far are *first-order*: they use only gradient information, which characterises only the local slope of C. *Second-order methods* use the Hessian $\nabla^2 C(\theta)$ (curvature information) to take steps that account for the local geometry of the loss landscape. This yields dramatically faster convergence near minima.

4.4.1 Newton's Method: Univariate Case



(a) Concave quadratic approximation: Newton step moves toward the local *maximum*.

(b) Convex quadratic approximation: Newton step moves toward the local *minimum*.

Figure 4.3: One Newton step. The sign of $C''(\theta^{(t)})$ determines whether we move toward a minimum or maximum.

[7] Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *ICLR 2017*.

For $\theta \in \mathbb{R}$, Newton's method[8] minimises the local quadratic approximation (second-order Taylor expansion) of C around the current iterate $\theta^{(t)}$:

$$C(\theta) \approx Q^{(t)}(\theta) = C(\theta^{(t)}) + (\theta - \theta^{(t)})C'(\theta^{(t)}) + \frac{(\theta - \theta^{(t)})^2}{2}C''(\theta^{(t)}).$$

Setting $(Q^{(t)})'(\theta) = 0$ gives the **Newton update**:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{C'(\theta^{(t)})}{C''(\theta^{(t)})}, \tag{4.18}$$

provided $C''(\theta^{(t)}) \neq 0$. If C is a quadratic, then $Q^{(t)} \equiv C$ and Newton's method reaches the critical point in *one step* from any starting point (this is the minimum when $C'' > 0$, otherwise the maximum).

Quadratic convergence. Close to a strict local minimum θ^* where $C''(\theta^*) \neq 0$, Newton's method converges *quadratically*: the error $e_t = |\theta^{(t)} - \theta^*|$ satisfies

$$e_{t+1} \leq \frac{|C'''(\theta^*)|}{2|C''(\theta^*)|}\, e_t^2 + O(e_t^3). \tag{4.19}$$

The constant comes from Taylor-expanding C' around θ^*; see the references cited above for the full derivation.

Quadratic convergence means the number of correct decimal digits *doubles* at each iteration. This is exponentially faster than GD's linear convergence.

Instabilities. Newton's method can overshoot, oscillate, or converge to the wrong critical point when:

- $C''(\theta^{(t)}) < 0$, in which case the Newton step heads toward a *maximum* rather than a minimum.
- $C''(\theta^{(t)})$ is near zero (inflection points), giving huge steps.
- The starting point is far from a minimum, so the quadratic approximation is poor.

Trust region methods limit the step size to a radius within which the quadratic approximation is trustworthy. A simpler safeguard is **damped Newton**: replace $C''(\theta^{(t)})$ in (4.18) by $C''(\theta^{(t)}) + \lambda$ for some $\lambda > 0$, interpolating between pure Newton ($\lambda \to 0$) and gradient descent ($\lambda \to \infty$). This is the univariate ancestor of the Levenberg-Marquardt algorithm for nonlinear least squares.

[8] Standard textbook treatment: Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.).

Secant method. When C'' is unavailable or expensive to compute, approximate it using the last two iterates:

$$C''(\theta^{(t)}) \approx \frac{C'(\theta^{(t)}) - C'(\theta^{(t-1)})}{\theta^{(t)} - \theta^{(t-1)}}. \tag{4.20}$$

The **secant method** substitutes (4.20) into (4.18) and requires only first derivatives. It converges *superlinearly* (rate ≈ 1.618, the golden ratio), between linear (GD) and quadratic (Newton).

Exercise 4.4.1 (Newton's Method: Convergence on an Explicit Function)

Let $C(\theta) = e^{\theta} - 2\theta$.

(a) Compute $C'(\theta)$ and $C''(\theta)$. Find the minimiser $\theta^* = \arg\min_\theta C(\theta)$ exactly. Starting from $\theta^{(0)} = 0$, perform two Newton steps using (4.18) and compute $\theta^{(1)}$ and $\theta^{(2)}$. Report the errors $e_t = |\theta^{(t)} - \theta^*|$ for $t = 0, 1, 2$ (you may use $\ln 2 \approx 0.693$ and $e^{-1} \approx 0.368$).

(b) Compute the quadratic-convergence constant $\kappa_C = |C'''(\theta^*)|/(2|C''(\theta^*)|)$ from (4.19). Compare the theoretical bound $e_1 \lesssim \kappa_C e_0^2$ with your observed e_1 from (a). Does the bound hold? Check also whether the ratio e_2/e_1^2 is closer to κ_C than e_1/e_0^2 is, and explain why.

(c) Starting from $\theta^{(0)} = 0$ and $\theta^{(1)} = 1$ (as computed in (a)), perform one secant step using (4.20) and (4.18). Report $\theta^{(2)}_{\text{secant}}$ and the corresponding error e_2^{secant}. Compare e_2^{secant} with the Newton error e_2 from (a) and comment on the convergence rate.

4.4.2 Newton's Method: Multivariate Case

For $\theta \in \mathbb{R}^d$, the second-order Taylor approximation of C at $\theta^{(t)}$ is

$$Q^{(t)}(\theta) = C(\theta^{(t)}) + (\theta - \theta^{(t)})^\top \nabla C(\theta^{(t)}) + \frac{1}{2}(\theta - \theta^{(t)})^\top H_t\,(\theta - \theta^{(t)}), \tag{4.21}$$

where $H_t = \nabla^2 C(\theta^{(t)})$ is the Hessian. Minimising $Q^{(t)}$ over θ gives $\nabla Q^{(t)}(\theta) = \nabla C(\theta^{(t)}) + H_t(\theta - \theta^{(t)}) = 0$, hence the **multivariate Newton update**:

$$\theta^{(t+1)} = \theta^{(t)} - H_t^{-1} \nabla C(\theta^{(t)}), \tag{4.22}$$

when H_t is non-singular. In practice the linear system $H_t\,\Delta\theta = -\nabla C(\theta^{(t)})$ is solved rather than computing H_t^{-1} explicitly.

Remark 4.4.1

If we replace H_t by the scalar matrix $(1/\alpha)I$ in (4.21), the quadratic approximation becomes $Q^{(t)}_{\text{GD}}(\theta) = C(\theta^{(t)}) + \nabla C(\theta^{(t)})^\top(\theta - \theta^{(t)}) + \frac{1}{2\alpha}\|\theta - \theta^{(t)}\|^2$, whose minimiser is exactly the GD update $\theta^{(t)} - \alpha \nabla C(\theta^{(t)})$. Hence gradient descent is Newton's method with a *spherical* (isotropic) Hessian approximation. Newton's method, using the true curvature, adapts to the actual ellipsoidal geometry of the loss landscape and requires no learning rate calibration.

Descent direction and safeguards. The Newton step $-H_t^{-1}\nabla C(\theta^{(t)})$ is a descent direction precisely when H_t is positive definite; when H_t is indefinite (near a saddle) or negative definite (near a maximum), Newton heads toward the wrong type of critical point, exactly as in the univariate case. As in the univariate setting (§4.4.1), the standard safeguard is **damped Newton**: replace H_t by $H_t + \lambda I$ for some $\lambda > 0$, which is guaranteed positive definite and interpolates between pure Newton ($\lambda \to 0$) and gradient descent ($\lambda \to \infty$). This is the multivariate Levenberg-Marquardt method.

Quadratic convergence. Near a strict local minimum θ^* where $\nabla^2 C(\theta^*)$ is positive definite and $\nabla^2 C$ is Lipschitz in a neighbourhood, Newton's method converges *quadratically*: $\|\theta^{(t+1)} - \theta^*\| \leq M \|\theta^{(t)} - \theta^*\|^2$ for some constant $M > 0$, generalising the univariate bound (4.19).

Computational cost. The Hessian H_t has d^2 entries, where $d = |\theta|$, so even *storing* it requires $O(d^2)$ memory. Solving the linear system $H_t\,\Delta\theta = -\nabla C(\theta^{(t)})$ via standard methods then costs $O(d^3)$. For neural networks with $d \sim 10^6$–10^9, the storage barrier alone (e.g. $d^2 \sim 10^{16}$ entries for $d = 10^8$) makes exact Newton completely infeasible. This motivates quasi-Newton methods.

4.4.3 Quasi-Newton Methods

Quasi-Newton methods maintain a sequence of positive definite matrices $B_0, B_1, B_2, \ldots$ where B_t approximates $H_t^{-1} = [\nabla^2 C(\theta^{(t)})]^{-1}$. The descent direction and update rule are:

$$d^{(t)} = -B_t \nabla C(\theta^{(t)}), \qquad \theta^{(t+1)} = \theta^{(t)} + \alpha^{(t)} d^{(t)}, \tag{4.23}$$

where $\alpha^{(t)} > 0$ is determined by a line search. The matrix B_t is updated at each iteration using gradient and step information.

The secant equation. A key constraint on B_t comes from the multivariate generalisation of the secant approximation (4.20). Define the step and gradient difference vectors:

$$\Delta^{(t)} = \theta^{(t+1)} - \theta^{(t)}, \qquad g_s^{(t)} = \nabla C(\theta^{(t+1)}) - \nabla C(\theta^{(t)}). \tag{4.24}$$

From the second-order Taylor approximation: $H_{t+1}\Delta^{(t)} \approx g_s^{(t)}$, or equivalently $B_{t+1} g_s^{(t)} \approx \Delta^{(t)}$. This is the **secant equation** (also called the quasi-Newton condition):

$$B_{t+1}\, g_s^{(t)} = \Delta^{(t)}. \tag{4.25}$$

The **curvature condition** ${g_s^{(t)}}^\top \Delta^{(t)} > 0$ guarantees existence of a positive definite B_{t+1} satisfying (4.25). It is enforced by using inexact line search with the **Wolfe conditions**[9].

[9] The Wolfe conditions bound the sufficient-decrease and curvature change during line search; see Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer, §3.1.

4.4.4 BFGS

The **Broyden-Fletcher-Goldfarb-Shanno (BFGS)** update rule[10] for B_t is:

$$B_{t+1} = V_t^\top B_t V_t + \rho^{(t)} \Delta^{(t)} \Delta^{(t)\top}, \tag{4.26}$$

where

$$\rho^{(t)} = \frac{1}{g_s^{(t)\top} \Delta^{(t)}}, \qquad V_t = I - \rho^{(t)}\, g_s^{(t)} \Delta^{(t)\top}. \tag{4.27}$$

The update is a rank-2 correction that preserves positive definiteness (guaranteed by the curvature condition) and satisfies the secant equation. It can be derived as the solution to: find the matrix B_{t+1} closest to B_t (in a weighted Frobenius norm) subject to $B_{t+1} = B_{t+1}^\top$ and the secant equation (4.25).

Cost. The matrix-matrix product $V_t^\top B_t V_t$ costs $O(d^2)$ per iteration as $d = |\theta|$, with $O(d^2)$ storage for B_t. This is feasible for small d but prohibitive for large-scale deep learning.

4.4.5 L-BFGS

Limited-memory BFGS (L-BFGS)[11] overcomes BFGS's memory and computational cost by *never explicitly forming* B_t. Instead, it stores only the latest m pairs $\{(\Delta^{(k)}, g_s^{(k)})\}_{k=t-m}^{t-1}$ and implicitly represents B_t through them.

Unrolling the BFGS recursion (4.26) writes B_t as a sandwich $V_{t-1}^\top \cdots V_0^\top\, B_0\, V_0 \cdots V_{t-1}$ around the initial B_0, plus a sum of t rank-one terms built from the pairs $(\Delta^{(k)}, g_s^{(k)})$ for $k = 0, \ldots, t-1$. L-BFGS keeps only the most recent m rank-one terms and *replaces the long V-sandwich around B_0* by a single symmetric positive-definite "initialisation" matrix $B_{\mathsf{init}}^{(t)}$, chosen anew at each iteration:

$$\widetilde{B}_t \;=\; V_{t-1}^\top \cdots V_{t-m}^\top\, B_{\mathsf{init}}^{(t)}\, V_{t-m} \cdots V_{t-1} \;+\; \sum_{k=t-m}^{t-1} \rho^{(k)}\, V_{t-1}^\top \cdots V_{k+1}^\top\, \Delta^{(k)} \Delta^{(k)\top}\, V_{k+1} \cdots V_{t-1}. \tag{4.28}$$

A standard practical choice is the scaled identity adapted to the most recent stored pair:

$$B_{\mathsf{init}}^{(t)} \;=\; \frac{g_s^{(t-1)\top} \Delta^{(t-1)}}{g_s^{(t-1)\top} g_s^{(t-1)}}\, I.$$

With $B_{\mathsf{init}}^{(t)}$ diagonal, the product $\widetilde{B}_t \nabla C(\theta^{(t)})$ can be computed in $O(md)$ via the **two-loop recursion**[12], using only vector dot products; here $d = |\theta|$.

[10] Independently proposed in four 1970 papers by Broyden, Fletcher, Goldfarb, and Shanno; for a unified treatment see Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer, Chapter 6.

[11] Liu, D. C., & Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. *Mathematical Programming* 45, 503–528.

[12] Nocedal, J. (1980). Updating quasi-Newton matrices with limited storage. *Mathematics of Computation* 35(151), 773–782.

Remark 4.4.2

Cost comparison. L-BFGS costs $O(md)$ per iteration in both time and memory, where $m \ll d$ (typically $m = 5$–20). This compares very favourably with BFGS's $O(d^2)$ and Newton's $O(d^3)$. For small-to-medium scale problems ($d \lesssim 10^5$), L-BFGS is often the method of choice due to its superlinear convergence. In deep learning, it is increasingly used to "polish" solutions found by Adam, or in federated learning settings where exact gradients are available.

Remark 4.4.3

Why second-order methods matter. For ill-conditioned problems (large condition number $\kappa = L/\mu$), GD needs $O(\kappa \log(1/\varepsilon))$ iterations to reach ε-accuracy, while Newton (near the minimum) needs only $O(\log\log(1/\varepsilon))$. This gap is enormous for realistic deep learning problems where κ can be 10^3–10^6. The current frontier of deep learning research is developing practical second-order methods for large-scale models.

Exercise 4.4.2 (Quasi-Newton: Secant Equation and Memory)

(a) Let $C(\theta) = \frac{1}{2}\theta^\top A\theta - b^\top\theta$ for a symmetric positive definite matrix $A \in \mathbb{R}^{d\times d}$. Show that for any step $\Delta^{(t)} = \theta^{(t+1)} - \theta^{(t)}$, the exact gradient difference satisfies $g_s^{(t)} = A\Delta^{(t)}$. Hence the secant equation (4.25) is satisfied if and only if $B_{t+1}A\Delta^{(t)} = \Delta^{(t)}$. What property of B_{t+1} does this require if it must hold for all $\Delta^{(t)}$?

(b) For the same strongly convex quadratic, verify that the curvature condition ${g_s^{(t)}}^\top \Delta^{(t)} > 0$ holds whenever $\Delta^{(t)} \neq 0$. What property of A makes this true?

(c) Consider a neural network with $d = 10^7$ parameters, stored as 32-bit floats (4 bytes each). Compare the memory required to store the BFGS inverse Hessian approximation $B_t \in \mathbb{R}^{d\times d}$ versus the L-BFGS history of $m = 10$ curvature pairs $\{(\Delta^{(k)}, g_s^{(k)})\}$. Express your answers in gigabytes (GB). Based on this, explain why BFGS is infeasible for modern deep learning while L-BFGS remains a viable option for moderate-scale problems.

4.5 Practical Comparison and Guidance

4.5.1 Early Stopping with Adaptive Optimisers

Recall from Week 2 §6 that **early stopping** monitors a validation performance $\mathcal{P}(\theta^{(t)})$ during training and returns the iterate with the best $\mathcal{P}(\theta^{(t)})$, even if the training loss $C(\theta^{(t)})$ continues to fall. With the aggressive adaptive optimisers of §4.3 (Adam, RMSProp), the validation optimum is typically reached within tens of epochs and then overshot rapidly while $C(\theta^{(t)})$ keeps decreasing, so monitoring $\mathcal{P}(\theta^{(t)})$ and stopping at the validation best becomes essential.

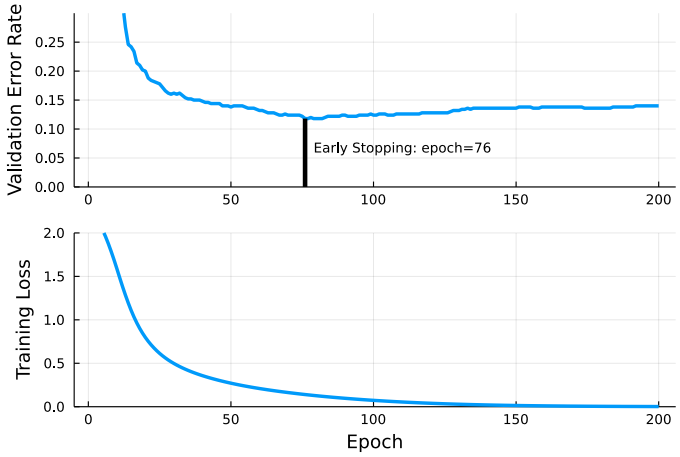


Figure 4.4: Training on a small MNIST subset (1000 training, 500 validation images). The best validation performance occurs at epoch 76, the early stopping point, after which validation error increases while training loss continues to fall.

4.5.2 Practical Guidance

- **Always use mini-batches**: For any first-order optimiser (SGD, NAG, Adam, RMSProp), replace the full gradient by a mini-batch estimate at each step; only L-BFGS and Newton are typically run full-batch, since their convergence properties degrade under noisy gradients.
- **Default choice**: Adam with learning rate $\alpha \in [10^{-4}, 10^{-3}]$, defaults $\beta = 0.9$, $\gamma = 0.999$. Add linear warm-up followed by cosine decay for better performance.
- **Batch size**: Larger n_b reduces gradient noise and enables better hardware utilisation, but is empirically associated with worse generalisation in deep learning (the *large-batch generalisation gap*). Typical values: $n_b \in \{32, 128, 512\}$.
- **Convex problems**: L-BFGS is often superior if d is not too large; NAG is the theoretically optimal first-order method.
- **Fine-tuning**: Pre-train with Adam (fast initial progress) then switch to SGD with a small fixed α for the final stretch; this practice is empirically associated with better generalisation than running Adam to convergence.
- **Monitoring**: Always track both training loss and validation performance. Use early stopping to guard against overtraining.